YOLOv8 Traffic Sign Detection – Colab Code

All code cells from the Colab notebook (YOLOv8n trained on a 6-class traffic sign dataset), in order, with a short note on each.

1. Install Ultralytics

Installs the Ultralytics package, which provides YOLOv8.

```
!pip install -q ultralytics
```

2. Check Ultralytics version

Confirms the install worked and prints the version (8.4.198 in this run).

```
import ultralytics

print("Ultralytics Version:", ultralytics.__version__)
```

3. Imports and GPU check

Imports the required libraries and checks that a CUDA GPU is available (Tesla T4 here).

```
import os
import shutil
import torch

from ultralytics import YOLO
from google.colab import drive

print("CUDA Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

4. Mount Google Drive

Mounts Google Drive so the dataset can be read and trained weights can be saved there.

```
drive.mount('/content/drive')
```

5. Set paths and create project folder

Defines the dataset zip path and creates the Drive folder where training results will be stored.

```
import os

ZIP_PATH = "/content/drive/MyDrive/Autonomous-Car.zip"

PROJECT_PATH = "/content/drive/MyDrive/YOLOv8_Project"

os.makedirs(PROJECT_PATH, exist_ok=True)

print("Project folder created!")
```

6. Extract the dataset

Unzips the dataset from Drive into the Colab local disk (faster to read during training).

```
import zipfile

EXTRACT_PATH = "/content/dataset"

with zipfile.ZipFile(ZIP_PATH, 'r') as zip_ref:
    zip_ref.extractall(EXTRACT_PATH)

print("Dataset extracted successfully!")
```

7. Find all YAML files

Searches the extracted folder for every `.yaml` file to locate the dataset configs.

```
import glob

yaml_files = glob.glob(
    "/content/dataset/**/*.yaml",
    recursive=True
)

print("YAML files found:")
for file in yaml_files:
    print(file)
```

8. Print the dataset YAML contents

Prints the three candidate dataset configs so you can compare classes and paths.

```
dataset_yaml_files = [
    "/content/dataset/Autonomous-Car/Traffic-Signs-new/data.yaml",
    "/content/dataset/Autonomous-Car/custom-traffic-
signs.v1i.yolov8/data.yaml",
    "/content/dataset/Autonomous-Car/AutoVehicle/data_custom.yaml"
]

for path in dataset_yaml_files:
    print("\n" + "="*60)
    print("FILE:", path)
    print("="*60)

    with open(path, "r") as f:
        print(f.read())
```

9. List folder contents of each dataset

Shows what files and folders sit next to each YAML (train/valid/test, README, etc.).

```
import os

for path in dataset_yaml_files:
    print("\nDataset:", path)

    folder = os.path.dirname(path)

    for item in os.listdir(folder):
        print("  ", item)
```

10. Rewrite the YAML with Colab paths

The original YAML had a Windows path, so this overwrites it with the correct Colab path and the 6 class names.

```
import yaml
import os

DATA_YAML = "/content/dataset/Autonomous-Car/Traffic-Signs-new/data.yaml"

DATASET_PATH = "/content/dataset/Autonomous-Car/Traffic-Signs-new"

data = {
    "path": DATASET_PATH,
    "train": "train/images",
    "val": "valid/images",
    "test": "test/images",
    "nc": 6,
    "names": [
        "left_turn",
        "straight_ahead",
        "right_turn",
        "dead_end",
        "no_entry",
        "stop"
    ]
}
```

```
with open(DATA_YAML, "w") as f:
    yaml.safe_dump(data, f, sort_keys=False)

print("Dataset YAML updated successfully!")
```

11. Verify image and label counts

Checks that every split has matching image and label counts (4200 / 1200 / 600).

```
for split in ["train", "valid", "test"]:

    image_path = os.path.join(
        DATASET_PATH, split, "images"
    )

    label_path = os.path.join(
        DATASET_PATH, split, "labels"
    )

    print(f"\n{split.upper()}")

    print("Images:", len(os.listdir(image_path)))
    print("Labels:", len(os.listdir(label_path)))
```

12. Confirm the updated YAML

Reads the YAML back to make sure the rewrite saved correctly.

```
with open(DATA_YAML, "r") as f:
    print(f.read())
```

13. Load YOLOv8 Nano

Loads the pretrained `yolov8n.pt` weights as the starting point for transfer learning.

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

print("YOLOv8 Nano loaded successfully!")
```

14. Train the model

Trains for 100 epochs at 640px, saving weights and plots to Drive (took about 1.9 hours on a T4).

```
results = model.train(
    data=DATA_YAML,
    epochs=100,
    imgsz=640,
    batch=16,
    device=0,
    workers=2,
    patience=0,
    optimizer="auto",
    project="/content/drive/MyDrive/YOLOv8_Project",
    name="traffic_signs_100_epochs",
    exist_ok=True,
    save=True,
    plots=True,
    verbose=True
)
```

15. Check saved weights

Confirms that `best.pt` and `last.pt` were saved in Drive.

```
BEST_MODEL =
"/content/drive/MyDrive/YOLOv8_Project/traffic_signs_100_epochs/weights/best.
pt"

LAST_MODEL =
"/content/drive/MyDrive/YOLOv8_Project/traffic_signs_100_epochs/weights/last.
pt"

print("Best model exists:", os.path.exists(BEST_MODEL))
print("Last model exists:", os.path.exists(LAST_MODEL))
```